

# Java Web 开发

---

RESTful API · 数据持久化 · 安全认证

1.47 亿条个人信息，在 78 天内被悄悄拿走.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

| 模块             | 关键技术                  |
|----------------|-----------------------|
| 构建 RESTful API | Spring Boot           |
| 安全操作数据库        | MyBatis + MySQL       |
| 用户认证与权限控制      | Spring Security + JWT |

## 开场案例：Equifax 数据泄露（2017）

"公司知道漏洞存在，却没有修补它。" ——美国国会听证会，2017

1.47 亿 美国公民的社会安全号（SSN）、出生日期、信用卡号遭窃

| 时间线        | 事件                               |
|------------|----------------------------------|
| 2017-03-08 | Apache Struts CVE-2017-5638 发布补丁 |
| 2017-05-13 | 攻击者开始入侵（利用未修复的漏洞）                |
| 2017-07-29 | Equifax 发现异常流量，入侵已持续 78 天        |
| 2017-09-07 | 公开披露数据泄露事件                       |
| 2019       | 赔偿 \$700 M（约 50 亿人民币）            |

Apache Struts 是一个广泛使用的 Java Web MVC 框架

## 根因分析：不只是"忘了打补丁"

⚠ 直接原因：CVE-2017-5638 — Content-Type Header 中的 OGNL 表达式注入，导致远程代码执行（RCE）

### 深层原因

- **依赖管理失控**：数百个 Java 库，没有自动漏洞扫描机制
- **认证授权缺失**：内部子系统之间无身份验证，入侵后横向移动畅通无阻
- **最小权限原则违反**：数据库账号权限过大，攻击者拿下 Web 层即可 `SELECT *` 全库

### 今天学什么？

Spring Boot + MyBatis + Spring Security — 正是解决这三个问题的现代 Java Web 技术栈

## 本讲路线图

| 模块     | 核心问题                | 关键技术                    | 时长      |
|--------|---------------------|-------------------------|---------|
| 基础     | Web 通信如何工作?         | HTTP · Tomcat · Servlet | ~15 min |
| 基础     | 如何设计现代 Web API?     | REST · JSON             | ~10 min |
| PART 1 | 如何快速构建 RESTful API? | Spring Boot             | ~20 min |
| PART 2 | 如何安全、高效地操作数据库?      | MyBatis                 | ~15 min |
| PART 3 | 如何实现用户认证与权限控制?      | Spring Security + JWT   | ~20 min |

三层技术栈，对应三道防线：接口层 → 数据层 → 安全层

---

# 基础知识

---

## | HTTP 协议

协议栈 · 请求响应 · Web 通信原理

## 协议栈：网络通信的分层模型

TCP/IP 是互联网的基础协议族，通常抽象为 4 层协议栈：

| 层次 | 名称    | 主要协议                 | 作用              |
|----|-------|----------------------|-----------------|
| 4  | 应用层   | HTTP · FTP · DNS     | 为应用程序提供网络服务接口   |
| 3  | 传输层   | TCP · UDP            | 端到端传输、差错控制、流量控制 |
| 2  | 网络层   | IP · ARP · ICMP      | 路由选择、拥塞控制、网络互连  |
| 1  | 网络接口层 | SLIP · PPP · IEEE802 | 物理上的可靠数据传输      |

HTTP 属于应用层，提供超文本数据的传输；它构建在 TCP 连接之上

## 数据包：层层封装

当应用层发送数据时，TCP/IP 逐层封装，接收方逐层解包：

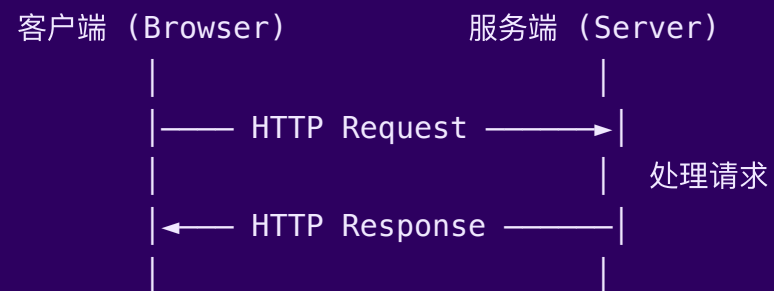
```
应用层  消息      | Hello World |  
      ↓  
传输层  段        | TCP/UDP 头 | Hello World |  
      ↓  
网络层  包        | IP 包头 | TCP/UDP 头 | Hello World |  
      ↓  
网络接口 帧      | MAC | LLC | IP 包头 | TCP/UDP 头 | Hello World | FCS |
```

- 数据可能被**拆分为多个包**依次发送
- 接收方需要对收到的数据进行**拆分或重组**



## HTTP 协议：客户端-服务器模型

HTTP (HyperText Transfer Protocol) 是应用层协议，采用 C/S 模型：



HTTP 无状态特性：

- 非 Keep-Alive 模式：每次请求建立新连接，响应后关闭，服务器不保留连接信息
- Keep-Alive 模式：Connection: Keep-Alive 头使连接持续有效，节省建连开销

# HTTP 工作流程

一次 HTTP 操作（事务）的完整过程：

## 1. 地址解析

URL: `http://localhost.com:8080/index.htm`

协议名      主机名      端口      路径

→ DNS 解析域名 → 获得服务器 IP 地址

## 2. 封装 HTTP 请求数据包

→ 结合本机信息，构建请求报文

## 3. 建立 TCP 连接（三次握手）

→ HTTP 默认端口 80，Spring Boot 开发常用 8080

## 4. 客户端发送请求

`GET /sample/hello.jsp HTTP/1.1`

## 5. 服务器响应

`HTTP/1.1 200 OK + 响应头 + 空行 + 响应体`

## 6. 关闭 TCP 连接（或 Keep-Alive 保持）

## URL：统一资源定位符

URL (Uniform Resource Locator) 是互联网上资源的标准地址，如同网络上的门牌号：

```
http://  blog.example.com  :8080  /users  ?gender=male&page=1
↑协议名  ↑主机名 (域名)    ↑端口  ↑路径  ↑查询参数 (键值对)
```

- 协议： `http` / `https` (加密)
- 主机名：通过 DNS 解析为服务器 IP 地址
- 端口：HTTP 默认 `80`，HTTPS 默认 `443`，Spring Boot 开发常用 `8080`
- 路径：服务器上的资源位置
- 查询参数：以 `?` 开始， `name=value` 格式，多个参数用 `&` 分隔

示例：

```
GET /api/products/123/reviews?page=2&size=10&sort=time HTTP/1.1
└─ 路径：层次化资源 ───┬── 过滤/分页参数 ───┘
```

## HTTP Request 格式

请求行: GET /users HTTP/1.1  
↑方法 ↑路径 ↑HTTP版本

Headers: Host: api.github.com  
Content-Type: text/plain  
Content-Length: 243

Body: body body body body... (GET 请求无 Body)

### 常用方法:

| 方法     | 用途             | 是否有 Body    |
|--------|----------------|-------------|
| GET    | 获取资源, 不修改服务器数据 | ✗           |
| POST   | 新增或修改资源        | ✓ JSON/表单数据 |
| PUT    | 更新资源 (替换整体)    | ✓           |
| DELETE | 删除资源           | ✗           |

## HTTP Response 格式与状态码

状态行: HTTP/1.1 200 OK  
↑版本 ↑状态码 ↑状态消息

Headers: Date: Sun, 10 Oct 2010 23:26:07 GMT  
Content-Type: text/html  
Content-Length: 12

(空行)  
Body: Hello world!

### 常用状态码:

| 范围  | 含义    | 典型示例                         |
|-----|-------|------------------------------|
| 2xx | 成功    | 200 OK · 201 Created         |
| 3xx | 重定向   | 301 永久移动 · 304 未修改           |
| 4xx | 客户端错误 | 400 请求错误 · 401 未认证 · 404 未找到 |
| 5xx | 服务器错误 | 500 服务器内部错误                  |

---

# 基础知识

---

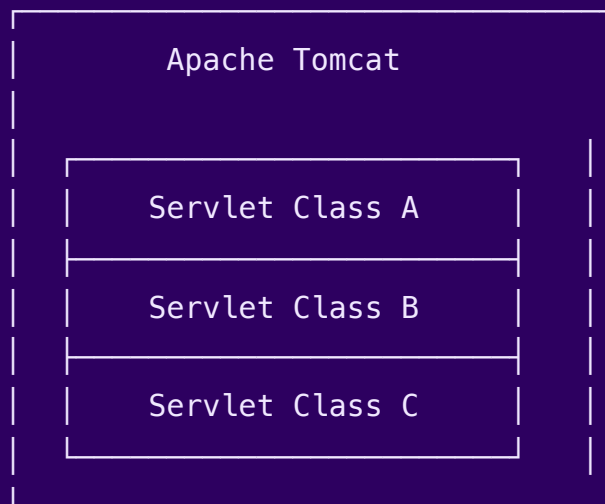
## | Tomcat 与 Servlet

Web 服务器 · 请求生命周期 · Java 动态内容

# Apache Tomcat: Java Web 服务器

Apache Tomcat 是用 Java 编写、运行在 JVM 上的 Web 服务器 (Java Servlet 容器):

Web 浏览器  
|  
| HTTP Request  
▼



HTTP Response  
▼  
Web 浏览器

## Tomcat 的核心职责

| 职责              | 说明                                                                                                 |
|-----------------|----------------------------------------------------------------------------------------------------|
| 管理 Servlet 生命周期 | 负责 Servlet 的加载、初始化、销毁                                                                              |
| URL 路由映射        | 将请求 URL 映射到对应的 Servlet 处理                                                                          |
| 请求/响应封装         | 将 HTTP 请求封装为 <code>HttpServletRequest</code> 对象，将 <code>HttpServletResponse</code> 转为 HTTP 响应返回浏览器 |

Spring Boot 已内嵌 Tomcat——`java -jar` 启动时，Tomcat 随之启动，无需手动部署



## | Servlet: Java 动态 Web 内容

Java Servlet 是用 Java 编写的服务器端程序，作为浏览器请求与后端数据库/应用之间的中间层：

Servlet 生命周期：

```
HTTP 请求到达
    ↓
Servlet 容器加载 Servlet 类
    ↓
    init()      ← 初始化（只执行一次）
    ↓
    service()   ← 每次请求触发（多线程并发）
    /      \
doGet() doPost() ← 根据 HTTP 方法分发
    ↓
    destroy()   ← 容器关闭时执行
```

## Servlet 代码示例

```
// 扩展 HttpServlet 类
public class HelloWorld extends HttpServlet {

    public void init() throws ServletException {
        // 初始化 (只执行一次)
        message = "Hello World";
    }

    // 处理 GET 请求
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>" + message + "</h1>");
    }

    public void destroy() { /* 清理资源 */ }
}
```

Spring Boot 的 `@RestController` 本质上就是对 Servlet 的高度封装——让我们无需手写 Servlet 样板代码

---

# 基础知识

---

## | REST 与 JSON

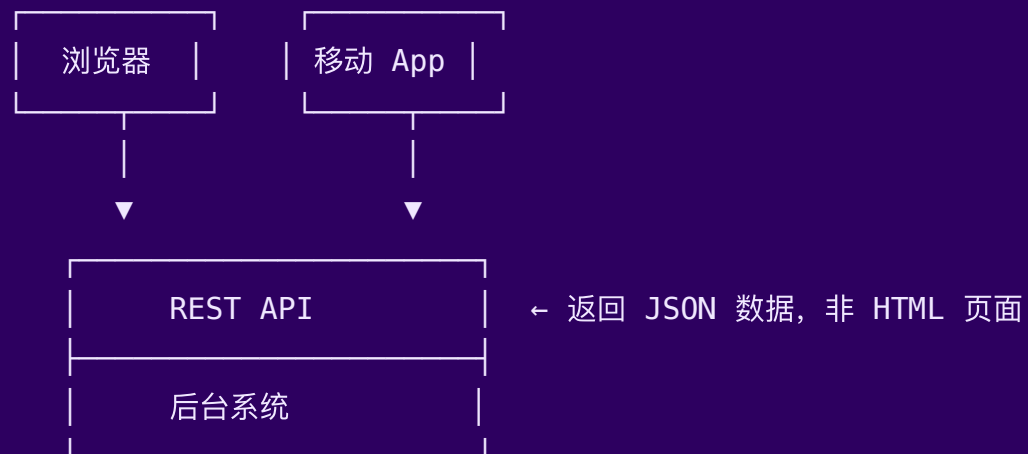
API 设计风格 · 前后端分离 · 数据格式

## REST 风格：现代 Web API 的标准

**REST** (Representational State Transfer) 由 Roy Fielding 博士在 2000 年博士论文中提出：

- REST 取代了复杂笨重的 SOAP，成为 **Web API 的事实标准**
- 核心特性：**无状态 · 统一接口 · 可扩展 · 分层系统**

前后端分离架构：



## REST API 规范：资源 + 方法

REST 以资源为中心设计 URL，用 HTTP 方法表达操作：

| 操作     | HTTP 方法 | URL 示例                                             |
|--------|---------|----------------------------------------------------|
| 获取全部商品 | GET     | /api/products                                      |
| 获取单个商品 | GET     | /api/products/123                                  |
| 新建商品   | POST    | /api/products                                      |
| 更新商品   | PUT     | /api/products/123                                  |
| 删除商品   | DELETE  | /api/products/123                                  |
| 获取商品评论 | GET     | /api/products/123/reviews                          |
| 分页查询评论 | GET     | /api/products/123/reviews?page=2&size=10&sort=time |

REST 不是强制标准，但遵守规范让 API 一目了然——URL 即文档

## JSON：轻量级数据交换格式

JSON (JavaScript Object Notation) 是 REST API 的标准数据格式：

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "city": "Nanjing",
    "state": "JS"
  },
  "phoneNumbers": [
    { "type": "mobile", "number": "138-0000-0000" },
    { "type": "work", "number": "025-8365-0000" }
  ]
}
```

JSON 数据类型： `string` · `number` · `object {}` · `array []` · `true/false` · `null`

MIME 类型： `application/json` ； 文件扩展名： `.json` ； 与语言无关，所有主流语言均支持

---

# PART 1

---

## | Spring Boot

约定优于配置 · 快速构建 RESTful API

## Spring Boot 是什么？

传统 Spring 项目需要手动编写数百行 XML 配置，Spring Boot 把这个工作消灭掉了。

| 特性         | 说明                                                 |
|------------|----------------------------------------------------|
| 约定优于配置     | 默认值覆盖 90% 场景，只需配置例外情况                              |
| 自动配置       | 检测 classpath，自动装配 DataSource、MVC 等组件               |
| Starter 依赖 | <code>spring-boot-starter-web</code> 一行引入 Web 全套依赖 |
| 内嵌服务器      | 打包为 fat JAR，直接 <code>java -jar</code> 运行，自带 Tomcat |

类比：传统 Spring 像散装零件自己组装，Spring Boot 像开箱即用的 MacBook——通电就能用



## 创建项目：Spring Initializr

访问 [start.spring.io](https://start.spring.io) 或使用 IntelliJ IDEA 内置向导

Project: Maven  
Language: Java  
Spring Boot: 3.x  
Java: 17+

Dependencies (今天全部需要):

- ✓ Spring Web ← RESTful API
- ✓ MyBatis Framework ← 数据库操作
- ✓ MySQL Driver ← 数据库驱动
- ✓ Spring Security ← 安全认证

## 生成的项目结构

```
src/main/java/com/example/  
├─ DemoApplication.java    ← 启动类 (@SpringBootApplication)  
├─ controller/             ← 控制器: 接收 HTTP 请求  
├─ service/                ← 服务层: 业务逻辑  
├─ mapper/                 ← 数据层: MyBatis Mapper 接口  
└─ entity/                 ← 实体类: 与数据库表对应
```

# Hello World: 第一个 RESTful API

```
// 启动类
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

// 控制器—处理 HTTP 请求, 返回 JSON
@RestController
public class SiteController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot!";
    }

    @GetMapping("/sites")
    public List<Map<String, Object>> getSites() {
        return List.of(
            Map.of("id", 1, "name", "南京大学", "url", "https://www.nju.edu.cn"),
            Map.of("id", 2, "name", "GitHub", "url", "https://github.com")
        );
    }
}
```

访问 <http://localhost:8080/sites> → 直接返回 JSON 数组

## 常用注解速查

| 注解                                  | 作用                   | 使用位置                      |
|-------------------------------------|----------------------|---------------------------|
| <code>@SpringBootApplication</code> | 启动入口，开启自动配置          | 启动类                       |
| <code>@RestController</code>        | 控制器，方法返回值自动序列化为 JSON | 类                         |
| <code>@GetMapping("/path")</code>   | 处理 HTTP GET 请求       | 方法                        |
| <code>@PostMapping("/path")</code>  | 处理 HTTP POST 请求      | 方法                        |
| <code>@RequestBody</code>           | 从请求体读取 JSON 并反序列化为对象 | 方法参数                      |
| <code>@PathVariable</code>          | 从 URL 路径提取变量         | <code>{id}</code> → 参数    |
| <code>@RequestParam</code>          | 从 URL 查询字符串读取值       | <code>?page=1</code> → 参数 |

`@RestController` = `@Controller` + `@ResponseBody`，两者等价

## 动手实践（5 分钟）

**任务：**创建并运行你的第一个 Spring Boot 项目

1. 打开 `start.spring.io`，勾选 `Spring Web`，生成并用 IDEA 导入项目
2. 创建 `HelloController`，添加 `@GetMapping("/hello")` 返回你的名字
3. 点击启动，在浏览器访问 `http://localhost:8080/hello`
4. **进阶：**添加 `@GetMapping("/info")` 返回包含 `name`、`studentId`、`time` 字段的 JSON 对象

✅ 成功标志：浏览器看到 JSON 响应，控制台无红色报错

💡 小贴士：`Map.of("key", value)` 可快速构造 JSON 对象；`List.of(...)` 可构造 JSON 数组

---

# PART 2

---

## | MyBatis

SQL 映射框架 · 告别手写 JDBC

## 为什么需要 MyBatis?

原始 JDBC 写法 (繁琐、危险):

```
Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM website WHERE id = ?");
ps.setInt(1, id);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    site.setName(rs.getString("name"));    // 手动映射每个字段
    site.setUrl(rs.getString("url"));
}
rs.close(); ps.close(); conn.close();    // 忘记关闭 → 连接泄漏!
```

MyBatis 写法 (简洁、安全):

```
@Mapper
public interface WebSiteMapper {
    @Select("SELECT * FROM website WHERE id = #{id}")
    WebSite findById(Integer id);
}
```

MyBatis 自动处理: 连接管理 · 参数预编译绑定 · 结果集映射 · 异常转换

## application.yml 数据源配置

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/demo?useSSL=false&serverTimezone=UTC
    username: root
    password: 123456
    driver-class-name: com.mysql.cj.jdbc.Driver

mybatis:
  mapper-locations: classpath:mapper/*.xml      # XML Mapper 文件路径
  type-aliases-package: com.example.entity      # 实体类包 (可省略全限定名)
  configuration:
    map-underscore-to-camel-case: true          # user_name → userName 自动映射
```

## 数据库建表

```
CREATE TABLE website (
  id      INT PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(100) NOT NULL,
  url     VARCHAR(255) NOT NULL
);
```

## MyBatis CRUD：注解写法

```
@Mapper
public interface WebSiteMapper {

    @Select("SELECT * FROM website")
    List<WebSite> findAll();

    @Select("SELECT * FROM website WHERE id = #{id}")
    WebSite findById(Integer id);

    @Insert("INSERT INTO website(name, url) VALUES(#{name}, #{url})")
    @Options(useGeneratedKeys = true, keyProperty = "id")
    int insert(WebSite site);

    @Update("UPDATE website SET name=#{name}, url=#{url} WHERE id=#{id}")
    int update(WebSite site);

    @Delete("DELETE FROM website WHERE id = #{id}")
    int delete(Integer id);
}
```

⚠ 安全提示： `#{id}` 是预编译参数（防 SQL 注入）， `${id}` 是字符串拼接（危险！永远别用于用户输入）



## 完整 MVC 分层结构

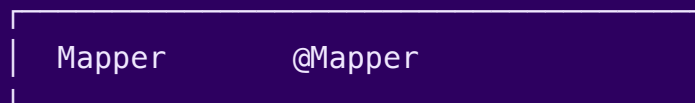
HTTP 请求



← 接收请求, 参数校验, 调用 Service



← 业务逻辑, 事务管理 (@Transactional)



← SQL 操作, 返回实体对象



MySQL 数据库

实体类 `WebSite` : `id`、`name`、`url` 字段与数据库列一一对应

## 停下来想一想

### MyBatis vs 全自动 ORM（如 Hibernate / JPA）——如何选择？

| 维度       | MyBatis       | Hibernate / JPA  |
|----------|---------------|------------------|
| SQL 控制粒度 | 完全手写，高度灵活     | 自动生成，可能产生低效 SQL  |
| 学习曲线     | 平缓，SQL 即代码    | 陡峭，需理解实体关系映射     |
| 复杂查询     | 擅长多表联查、动态 SQL | 复杂场景易出现 N+1 查询问题 |
| 国内生态     | 国内大厂主流选型      | 国际项目、DDD 场景更常见   |

**思考题：**回到 Equifax 案例——攻击者攻破 Web 层后，为什么能查到 1.47 亿条数据？如果数据库账号遵循最小权限原则，后果会有何不同？

---

# PART 3

---

## | Spring Security

认证与授权 · JWT Token · 守护你的每一个接口

## 认证 vs 授权

| 概念 | 英文                     | 要回答的问题 | 举例         |
|----|------------------------|--------|------------|
| 认证 | Authentication (AuthN) | 你是谁?   | 用户名 + 密码登录 |
| 授权 | Authorization (AuthZ)  | 你能做什么? | 管理员才能删除用户  |

### Spring Security 的核心：过滤器链 (Filter Chain)

HTTP 请求进来

- └─ UsernamePasswordAuthenticationFilter ← 处理表单登录
- └─ JwtAuthenticationFilter ← 验证 Token (自定义)
- └─ ExceptionTranslationFilter ← 将异常转为 401 / 403 响应
- └─ FilterSecurityInterceptor ← 最终权限检查

|

▼

Controller (只有通过全部过滤器才能到达)

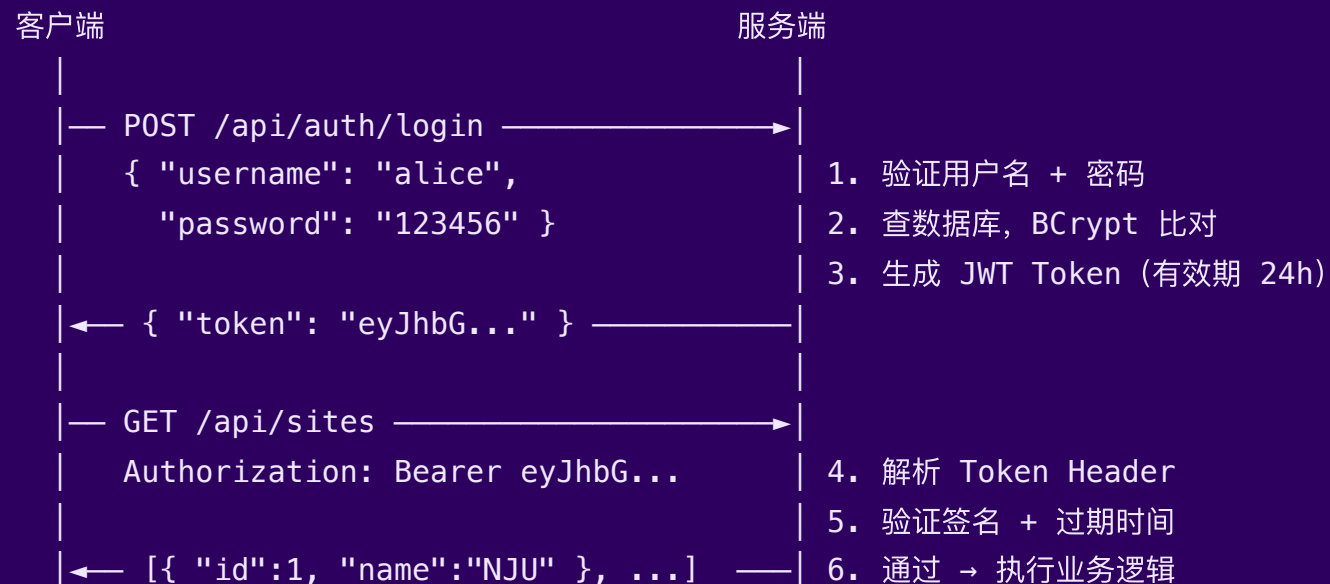
核心设计：安全横切关注点——不侵入业务代码，全部在过滤器链中处理

## SecurityFilterChain 配置

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http,
                                           JwtAuthFilter jwtFilter)
        throws Exception {
        http
            .csrf(csrf -> csrf.disable())           // REST API 场景关闭 CSRF
            .sessionManagement(s ->
                s.sessionCreationPolicy(STATELESS)) // JWT 无状态, 不使用 Session
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()    // 登录/注册不需要认证
                .requestMatchers("/api/admin/**").hasRole("ADMIN") // 管理接口需要角色
                .anyRequest().authenticated()                   // 其余接口均需认证
            )
            .addFilterBefore(jwtFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

# JWT 认证流程



JWT 结构: `Header.Payload.Signature` (三段 Base64 用 `.` 分隔)

`eyJhbGciOiJIUzI1NiJ9` ← Header: 签名算法 (HS256)  
`.eyJ1c2VyIjoiYWxpY2UiLCJleHAiOi0jF9` ← Payload: 用户信息 (Base64 编码, 非加密! )  
`.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6` ← Signature: 密钥 HMAC 签名, 防篡改

⚠ JWT Payload 只是 Base64 编码, 任何人都能解码——绝不要把密码存入 Payload!

## UserDetailsService: 连接 Security 与数据库

```
@Service
public class UserDetailsServiceImpl implements UserDetailsService {

    @Autowired
    private UserMapper userMapper;    // MyBatis Mapper, 从数据库查用户

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        User user = userMapper.findByUsername(username);
        if (user == null) {
            throw new UsernameNotFoundException("用户不存在: " + username);
        }
        // 构建 Spring Security 用户对象, 供框架进行密码比对
        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword())    // 数据库存的是 BCrypt 哈希值
            .roles(user.getRole())          // 角色: USER / ADMIN
            .build();
    }
}
```

## 登录与注册接口

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

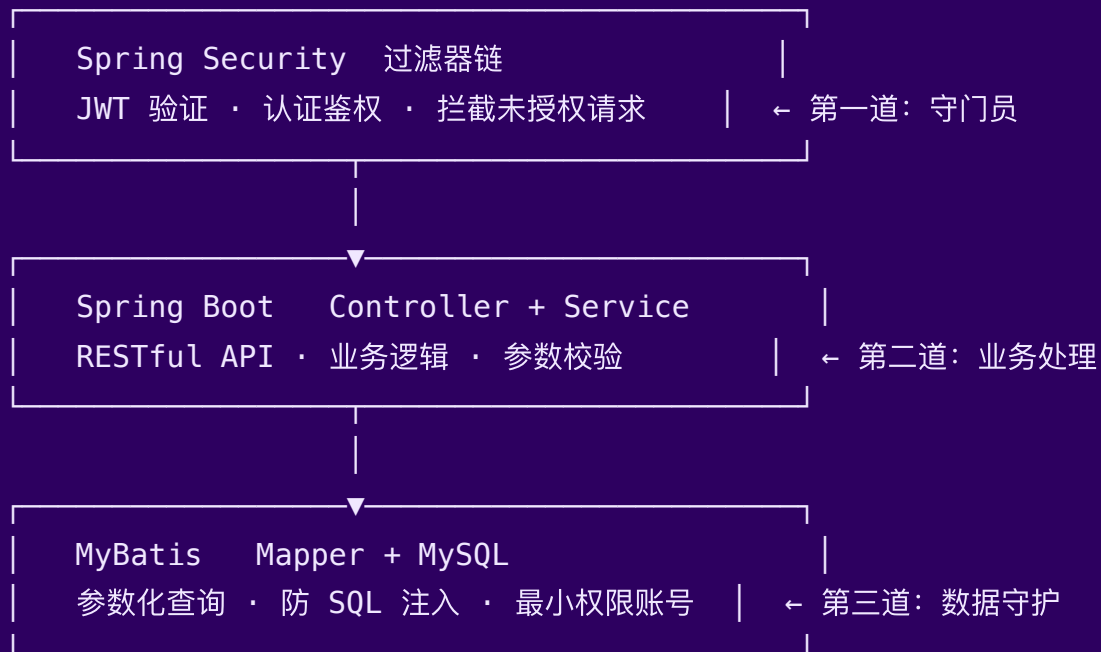
    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        // 1. 委托 Spring Security 验证用户名 + 密码
        Authentication auth = authManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                req.getUsername(), req.getPassword()));
        // 2. 验证通过, 生成 JWT Token (有效期 24h)
        String token = jwtUtil.generateToken(auth.getName());
        return ResponseEntity.ok(Map.of("token", token));
    }

    @PostMapping("/register")
    public ResponseEntity<?> register(@RequestBody RegisterRequest req) {
        // 3. 注册时 BCrypt 加密密码再存入数据库
        String encoded = passwordEncoder.encode(req.getPassword());
        userMapper.insert(new User(req.getUsername(), encoded, "USER"));
        return ResponseEntity.ok(Map.of("message", "注册成功"));
    }
}
```



## 课堂总结：三层技术栈 = 三道防线

Browser / Postman / 移动 App



**回到 Equifax：**若当年使用 Spring Security（及时升级框架）+ MyBatis 参数化查询 + 最小权限数据库账号，1.47 亿条数据不会如此轻易被窃

## 作业（二选一，选做加分）

### 选题 A：网站收藏夹后端 API

基于 Spring Boot + MyBatis 实现完整 CRUD：

- GET /api/sites 查询全部； POST /api/sites 新增（校验 url 格式）
- PUT /api/sites/{id} 修改； DELETE /api/sites/{id} 删除
- 用 Postman 测试所有接口，截图 + 代码提交

### 选题 B：用户认证系统

基于 Spring Security + JWT 实现：

- 用户注册（BCrypt 加密存储密码）
- 用户登录（返回 JWT Token）
- 受保护接口（需携带有效 Token 才能访问）
- Postman 演示完整认证流程，截图提交

加分项：两题结合——在 A 的 CRUD 接口上加 B 的登录鉴权保护（DELETE 需要 ADMIN 角色）

## 三分钟回顾

| 问题                                                            | 答案                                           |
|---------------------------------------------------------------|----------------------------------------------|
| Spring Boot 的核心优势?                                            | 自动配置 + Starter 依赖 + 内嵌 Tomcat, 开箱即用          |
| <code>@RestController</code> vs <code>@Controller</code> 的区别? | 前者默认带 <code>@ResponseBody</code> , 直接返回 JSON |
| MyBatis 中 <code>#{id}</code> vs <code>\${id}</code> 的区别?      | 前者预编译参数 (安全), 后者字符串拼接 (危险!)                  |
| JWT 的三个组成部分?                                                  | Header (算法) + Payload (数据) + Signature (签名)  |
| 密码如何存储?                                                       | BCrypt 加密哈希, <b>绝不存明文</b>                    |
| Spring Security 核心机制?                                         | 过滤器链 (Filter Chain) —— 安全逻辑与业务逻辑解耦           |

## 下节预告

### 第 6 讲：项目级 VibeCoding

用自然语言驱动开发——Stitch 设计原型、AI Studio 生成前端、Claude Code 生成后端，从零搭建一个完整全栈应用.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

## 第 5 讲 Java Web 开发

本讲核心收获

- Spring Boot: 5 分钟搭建 RESTful API, 注解驱动、自动配置
- MyBatis: 参数化 SQL 映射, 彻底告别手写 JDBC 连接泄漏
- Spring Security: 过滤器链 + JWT, 构建无状态安全认证体系

Spring Boot · MyBatis · Spring Security